Pass data between destinations

Navigation allows you to attach data to a navigation operation by defining arguments for a destination. For example, a user profile destination might take a user ID argument to determine which user to display.

In general, you should strongly prefer passing only the minimal amount of data between destinations. For example, you should pass a key to retrieve an object rather than passing the object itself, as the total space for all saved states is limited on Android.

Define destination arguments

To pass data between destinations, first define the argument by adding it to the destination that receives it by following these steps:

1. In the Navigation editor, click on the destination that receives the argument.
2. In the **Attributes** panel, click **Add** (**+**).
3. In the **Add Argument Link** window that appears, enter the argument name, argument type, whether the argument is nullable, and a default value, if needed.
4. Click **Add**. Notice that the argument now appears in the **Arguments** list in the **Attributes** panel.
5. Next, click on the corresponding action that takes you to this destination. In the **Attributes** panel, you should now see your newly added argument in the **Argument Default Values** section.
6. You can also see that the argument was added in XML. Click the **Text** tab to toggle to XML view, and notice that your argument was added to the destination that receives the argument. An example is shown below:

```
<fragment android:id="@+id/myFragment" >
  <argument
    android:name="myArg"
    app:argType="integer"
    android:defaultValue="0" />
</fragment>
```

Supported argument types

The Navigation library supports the following argument types:

Type	app:argType syntax	Support for default values	Handled by routes	Nullable
Integer	app:argType="integer"	Yes	Yes	No
Float	app:argType="float"	Yes	Yes	No
Long	app:argType="long"	Yes - Default values must always end with an 'L' suffix (e.g. "123L").	Yes	No
Boolean	app:argType="boolean"	Yes - "true" or "false"	Yes	No
String	app:argType="string"	Yes	Yes	Yes
Resource Reference	app:argType="reference"	Yes - Default values must be in the form of "@resourceType/resourceName" (e.g. "@style/myCustomStyle") or "0"	Yes	No
Custom Parcelable	app:argType="<type>", where <type> is the fully-qualified class name of the Parcelable	Supports a default value of "@null". Does not support other default values.	No	Yes

Custom Serializable	app:argType="<type>", where <type> is the fully-qualified class name of the Serializable	Supports a default value of "@null". Does not support other default values.	No	Yes
Custom Enum	app:argType="<type>", where <type> is the fully-qualified name of the enum	Yes - Default values must match the unqualified name (e.g. "SUCCESS" to match MyEnum.SUCCESS).	No	No

Note: References to resources are supported *only* in reference types. Using a resource reference in any other type results in an exception.

If an argument type supports null values, you can declare a default value of null by using android:defaultValue="@null".

Routes, deep links, and URIs with their arguments can be parsed from strings. This is not possible using custom data types such as Parcelables and Serializables as seen in the above table. To pass around custom complex data, store the data elsewhere such as a ViewModel or database and only pass an identifier while navigating; then retrieve the data in the new location after navigation has concluded.

When you choose one of the custom types, the **Select Class** dialog appears and prompts you to choose the corresponding class for that type. The **Project** tab lets you choose a class from your current project.

You can choose **<inferred type>** to have the Navigation library determine the type based on the provided value.

You can check **Array** to indicate that the argument should be an array of the selected **Type** value. Note the following:

> Arrays of enums and arrays of resource references are not supported.
> Arrays support nullable values, regardless of the support for nullable values of the underlying type. For example, using app:argType="integer[]" allows you to use app:nullable="true" to indicate that passing a null array is acceptable.
> Arrays support a single default value, "@null". Arrays do not support any other default value.

Caution: Passing complex data structures over arguments is considered an anti-pattern. Each destination should be responsible for loading UI data based on the minimum necessary information, such as item IDs. This simplifies process recreation and avoids potential data inconsistencies.

Override a destination argument in an action

Destination-level arguments and default values are used by all actions that navigate to the destination. If needed, you can override the default value of an argument (or set one if it doesn't already exist) by defining an argument at the action level. This argument must be of the same name and type as the argument declared in the destination.

The XML below declares an action with an argument that overrides the destination-level argument from the example above:

```
<action android:id="@+id/startMyFragment"
  app:destination="@+id/myFragment">
  <argument
    android:name="myArg"
    app:argType="integer"
    android:defaultValue="1" />
```

</action>

Use Safe Args to pass data with type safety

The Navigation component has a Gradle plugin called Safe Args that generates simple object and builder classes for type-safe navigation and access to any associated arguments. Safe Args is strongly recommended for navigating and passing data, because it ensures type-safety.

In some cases, for example if you are not using Gradle, you can't use the Safe Args plugin. In these cases, you can use Bundles to directly pass data.

To add Safe Args to your project, include the following classpath in your top level build.gradle file:

```
buildscript {
  repositories {
    google()
  }
  dependencies {
    def nav_version = "2.4.1"
    classpath "androidx.navigation:navigation-safe-args-gradle-plugin:$nav_version"
  }
}
```

You must also apply one of two available plugins.

To generate Java language code suitable for Java or mixed Java and Kotlin modules, add this line to **your app or module's** build.gradle file:

```
plugins {
 id 'androidx.navigation.safeargs'
}
```

Alternatively, to generate Kotlin code suitable for Kotlin-only modules add:

```
plugins {
 id 'androidx.navigation.safeargs.kotlin'
}
```

You must have android.useAndroidX=true in your gradle.properties file.

After enabling Safe Args, your generated code contains the following type-safe classes and methods for each action as well as with each sending and receiving destination.

> A class is created for each destination where an action originates. The name of this class is the name of the originating destination, appended with the word "Directions". For example, if the originating destination is a fragment that is named SpecifyAmountFragment, the generated class would be called SpecifyAmountFragmentDirections.
>
> This class has a method for each action defined in the originating destination.
>
> For each action used to pass the argument, an inner class is created whose name is based on the action. For example, if the action is called confirmationAction, the class is named ConfirmationAction. If your action contains arguments without a defaultValue, then you use the associated action class to set the value of the arguments.
>
> A class is created for the receiving destination. The name of this class is the name of the destination, appended with the word "Args". For example, if the destination fragment is

named ConfirmationFragment, the generated class is called ConfirmationFragmentArgs. Use this class's fromBundle() method to retrieve the arguments.

The following example shows you how to use these methods to set an argument and pass it to the navigate() method:

```
@Override
public void onClick(View view) {
 EditText amountTv = (EditText) getView().findViewById(R.id.editTextAmount);
 int amount = Integer.parseInt(amountTv.getText().toString());
 ConfirmationAction action =
     SpecifyAmountFragmentDirections.confirmationAction();
 action.setAmount(amount);
 Navigation.findNavController(view).navigate(action);
}
```

In your receiving destination's code, use the getArguments() method to retrieve the bundle and use its contents. When using the -ktx dependencies, Kotlin users can also use the by navArgs() property delegate to access arguments.

```
@Override
public void onViewCreated(View view, @Nullable Bundle savedInstanceState) {
  TextView tv = view.findViewById(R.id.textViewAmount);
  int amount = ConfirmationFragmentArgs.fromBundle(getArguments()).getAmount();
  tv.setText(amount + "");
}
```

Use Safe Args with a global action

When using Safe Args with a global action, you must provide an android:id value for your root <navigation> element, as shown in the following example:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:app="http://schemas.android.com/apk/res-auto"
      xmlns:tools="http://schemas.android.com/tools"
      xmlns:android="http://schemas.android.com/apk/res/android"
      android:id="@+id/main_nav"
      app:startDestination="@id/mainFragment">
  ...
</navigation>
```

Navigation generates a Directions class for the <navigation> element that is based on the android:id value. For example, if you have a <navigation> element with android:id=@+id/main_nav, the generated class is called MainNavDirections. All destinations within the <navigation> element have generated methods for accessing all associated global actions using the same methods as described in the previous section.

Pass data between destinations with Bundle objects

If you aren't using Gradle, you can still pass arguments between destinations by using Bundle objects. Create a Bundle object and pass it to the destination using navigate(), as shown below:

```
Bundle bundle = new Bundle();
bundle.putString("amount", amount);
Navigation.findNavController(view).navigate(R.id.confirmationAction, bundle);
```

In your receiving destination's code, use the getArguments() method to retrieve the Bundle and use its contents:

```
TextView tv = view.findViewById(R.id.textViewAmount);
```

```
tv.setText(getArguments().getString("amount"));
```

Pass data to the start destination

You can pass data to your app's start destination. First, you must explicitly construct a Bundle that holds the data. Next, use one of the following methods to pass the Bundle to the start destination:

- If you're creating your NavHost programmatically, call NavHostFragment.create(R.navigation.graph, args), where *args* is the Bundle that holds your data.
- Otherwise, you can set start destination arguments by calling one of the following overloads of NavController.setGraph():
  - Using the ID of the graph: navController.setGraph(R.navigation.graph, args)
  - Using the graph itself: navController.setGraph(navGraph, args)

To retrieve the data in your start destination, call Fragment.getArguments().

Note: when manually calling setGraph() with arguments, you must **not** use the app:navGraph attribute when creating the NavHostFragment in XML as that will internally call setGraph() without any arguments, resulting in your graph and start destination being created twice.